



“Año de la Esperanza y el Fortalecimiento de la Democracia”

CARRERA: Ingeniería de Sistemas e Informática

CURSO: Herramientas de Desarrollo

TÍTULO DEL PROYECTO:

UrbanFlow – Sistema de Gestión de Pedidos e Inventario de Ropa

CICLO: Séptimo

SEDE: San Juan de Lurigancho

DOCENTE: José Luis Milla Flores

INTEGRANTES: Grupo 08

- Christian Rivas Aquino (U23226966)
- Uriel Vásquez Quispe (U23227010)
- Renzo Estefano Yupanqui De la Cruz (U22229407)
- David Jair Mendoza Figueroa (U22226721)
- Aldair Ronaldo Casimiro Ramos (U23252961)

AÑO: 2026

Link de exposición: <https://www.youtube.com/watch?v=ISIMbo20EJI>

Índice

HITO 1

1. Fecha de inicio y finalización
2. Agradecimiento y Dedicatoria
3. Resumen y Abstract
4. Realidad Problemática y Justificación del proyecto
5. Objetivo general y específicos
6. Marco teórico y Metodología
7. Cronograma de actividades (diagrama de Gantt)
8. Desarrollo del proyecto (Aplicación y Base de datos)
9. Herramientas Git y GitHub
10. Resultados y Palabras clave
11. Conclusiones

Hito 1 (Semana 5 - APF1): Fundamentos, Arquitectura y Gobierno de Código

1. Fecha de inicio y finalización

- Inicio: (07/04/2026)
- Finalización: (28/04/2026)

2. Agradecimiento y Dedicatoria

Agradecimiento

Expresamos nuestro sincero agradecimiento al docente del curso por su orientación constante, sus valiosas recomendaciones técnicas y la retroalimentación brindada durante cada etapa del desarrollo del proyecto. Asimismo, agradecemos a la universidad por proporcionarnos el entorno académico, las salas de cómputo y los recursos necesarios para fortalecer nuestras competencias en la Ingeniería de Sistemas e Informática. Finalmente, extendemos nuestro agradecimiento a nuestras familias por su apoyo incondicional, comprensión y motivación permanente a lo largo de este proceso.

Dedicatoria

Dedicamos este proyecto a nuestras familias, quienes han sido un pilar fundamental en nuestra formación. También lo dedicamos a nuestros compañeros y docentes, quienes contribuyeron con su conocimiento y colaboración al logro de este trabajo.

3. Resumen y Abstract

Resumen

UrbanFlow es un sistema de gestión de pedidos e inventario orientado a una tienda de ropa, cuyo objetivo principal es optimizar los procesos de registro de productos, control de stock y gestión de pedidos de clientes. El sistema fue desarrollado utilizando Spring Boot como framework backend y MySQL como sistema gestor de base de datos relacional. Se implementa una arquitectura en capas (Controller, Service, Repository) y una API REST para la comunicación entre cliente y servidor.

El sistema permite realizar operaciones CRUD sobre los productos, registrar pedidos con cálculo automático de totales y actualizar el inventario en tiempo real. Además, facilita la consulta de información mediante endpoints estructurados, lo que mejora la trazabilidad de las operaciones y reduce los errores generados por la gestión manual.

Abstract

UrbanFlow is an order and inventory management system for a clothing store. It is designed to optimize product registration, stock control, and customer order processing. The system is developed using Spring Boot as backend framework and MySQL as relational database. It follows a layered architecture and exposes RESTful APIs.

The application supports CRUD operations for products, automatic total calculation for orders, and real-time inventory updates. It improves operational efficiency, data consistency, and reduces human errors.

4. Realidad Problemática y Justificación del proyecto

Realidad Problemática

En la actualidad, muchas pequeñas y medianas empresas del sector retail, especialmente las tiendas de ropa gestionan sus procesos mediante registros manuales o herramientas no integradas, como hojas de cálculo, cuadernos y aplicaciones separadas. Esta situación genera diversos problemas, entre ellos inconsistencias en los datos, pérdida de información, dificultad para conocer el stock real, errores en el cálculo de ventas y limitaciones al momento de generar reportes.

Asimismo, la falta de integración entre las ventas y el inventario dificulta la toma de decisiones oportunas, afectando directamente la rentabilidad del negocio. Por ello, surge la necesidad de implementar soluciones tecnológicas que permitan automatizar y centralizar la información de manera eficiente.

Justificación

El desarrollo de UrbanFlow se justifica en la necesidad de digitalizar y optimizar los procesos internos de una tienda de ropa. Mediante el uso de herramientas modernas como Spring Boot y MySQL, se propone una solución escalable, segura y eficiente.

Este sistema permite reducir errores humanos, mejorar la precisión de los datos, automatizar el control de inventario y facilitar la gestión de pedidos. Asimismo, contribuye a la formación académica de los estudiantes al aplicar conocimientos de desarrollo de software, bases de datos y arquitectura de sistemas.

5. Objetivo general y específicos

Objetivo General

Desarrollar un sistema web de gestión de pedidos e inventario para una tienda de ropa, utilizando herramientas modernas de desarrollo que permitan mejorar la eficiencia operativa y el control de la información.

Objetivos Específicos

- Diseñar un modelo de base de datos relacional que represente correctamente las entidades del sistema (productos, pedidos y clientes).
- Implementar operaciones CRUD para la gestión de productos.
- Desarrollar el módulo de pedidos con cálculo automático de totales.
- Implementar el control de inventario en tiempo real.
- Integrar el backend desarrollado en Spring Boot con la base de datos MySQL.
- Exponer funcionalidades mediante una API REST.
- Realizar pruebas funcionales para validar el correcto funcionamiento del sistema.

6. Marco teórico y Metodología

Marco Teórico

El desarrollo del proyecto Urban Flow se fundamenta en diversos conceptos clave de la Ingeniería de Software, los cuales permiten garantizar una solución eficiente, escalable y mantenible.

- **Spring Boot:**

Framework del ecosistema Java que facilita el desarrollo de aplicaciones web y servicios backend, reduciendo la configuración inicial y permitiendo la creación rápida de APIs REST robustas.

- **API REST:**

Estilo arquitectónico basado en el protocolo HTTP que permite la comunicación entre cliente y servidor mediante operaciones como GET, POST, PUT y DELETE, favoreciendo la interoperabilidad entre sistemas.

- **MySQL:**

Sistema de gestión de bases de datos relacional que permite almacenar, consultar y administrar grandes volúmenes de información estructurada de manera eficiente.

- **Arquitectura en capas:**

Modelo de diseño que divide la aplicación en diferentes niveles: presentación, lógica de negocio y acceso a datos, lo que mejora la organización del código, facilita el mantenimiento y permite la escalabilidad del sistema.

- **JPA/Hibernate:**

Tecnologías de mapeo objeto-relacional (ORM) que permiten convertir clases Java en tablas de base de datos, simplificando la persistencia y manipulación de datos.

- **Metodologías ágiles (Scrum):**

Scrum es un marco de trabajo ágil que permite desarrollar software de forma iterativa e

incremental, organizando el trabajo en periodos cortos llamados sprints. Facilita la colaboración del equipo, la adaptación a cambios y la entrega continua de funcionalidades.

Metodología

Para el desarrollo del sistema se utilizó la metodología ágil Scrum, la cual permitió organizar el trabajo en iteraciones cortas (sprints), facilitando el seguimiento del progreso y la mejora continua del proyecto.

El equipo estuvo conformado por cinco integrantes, quienes trabajaron de manera colaborativa, dividiendo las tareas según roles y responsabilidades, lo que permitió avanzar de forma paralela y eficiente.

Fases del desarrollo (Sprints)

- **Sprint 1 (Semana 3):**

Análisis de requerimientos y definición del alcance del sistema.

- **Sprint 2 (Semana 4):**

Diseño de la base de datos y estructura del sistema.

- **Sprint 3 (Semana 5):**

Desarrollo del backend (controladores, servicios y conexión con la base de datos).

- **Sprint 4 (Semana 6):**

Pruebas funcionales, validación del sistema y documentación final.

Distribución del equipo

- **Integrante 1:** Diseño y modelado de la base de datos
- **Integrante 2:** Desarrollo de controladores (API REST)
- **Integrante 3:** Implementación de la lógica de negocio (servicios)
- **Integrante 4:** Pruebas funcionales y validación del sistema
- **Integrante 5:** Documentación, apoyo general y control del proyecto

7. Cronograma de actividades (diagrama de Gantt)

Actividad	Semana 3	Semana 4	Semana 5	Semana 6
Análisis	✓			
Diseño BD	✓	✓		
Desarrollo Backend		✓	✓	
Pruebas			✓	✓
Documentación			✓	✓
Entrega				✓

8. Desarrollo del proyecto (Aplicación y Base de datos)

Arquitectura

El sistema sigue una arquitectura en capas:

- Controller: recibe y gestiona las solicitudes HTTP.
- Service: contiene la lógica de negocio.
- Repository: interactúa con la base de datos.

API REST

Se implementaron endpoints para la gestión de recursos:

- GET /api/productos
- POST /api/productos
- GET /api/pedidos
- POST /api/pedidos

Base de Datos

Se diseñó una base de datos relacional con las siguientes tablas:

- productos (id, nombre, precio, stock)
- clientes (id, nombre, correo)
- pedidos (id, fecha, total)
- detalle_pedido (id, pedido_id, producto_id, cantidad)

9. Herramientas Git y GitHub

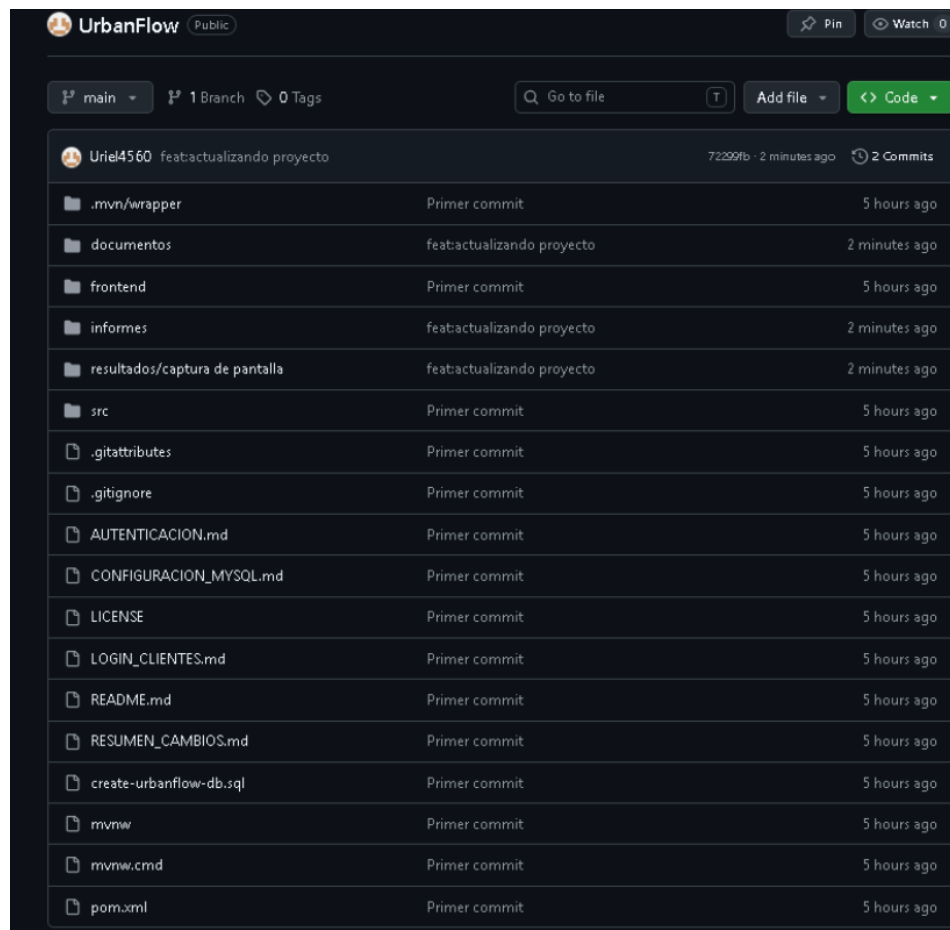
Git

Es un sistema de control de versiones que permite guardar cambios en el código, trabajar en distintas versiones (ramas) y volver atrás si algo falla.

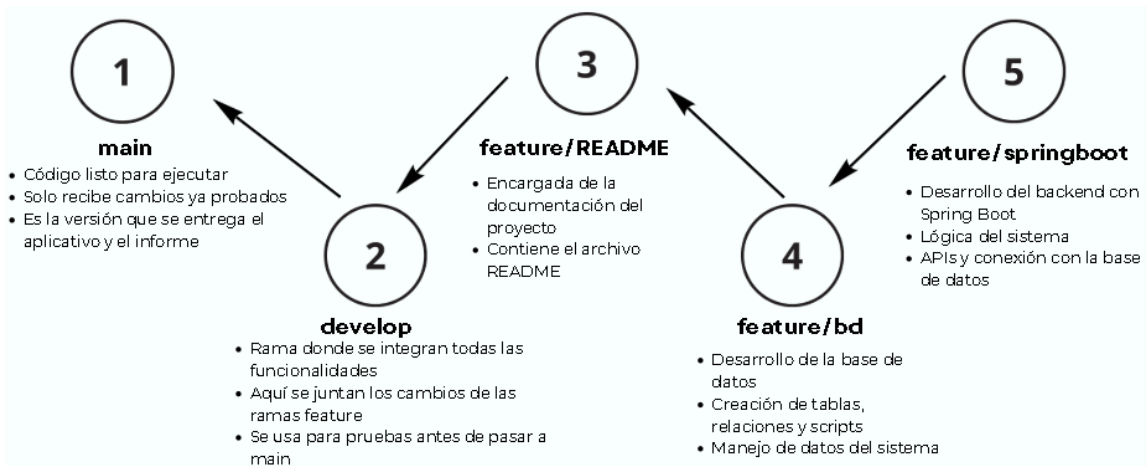
GitHub

Es una plataforma en la nube donde se alojan repositorios Git. Facilita el trabajo en equipo y la colaboración mediante herramientas como Pull Requests.

Estructura del repositorio



Organización del Repositorio



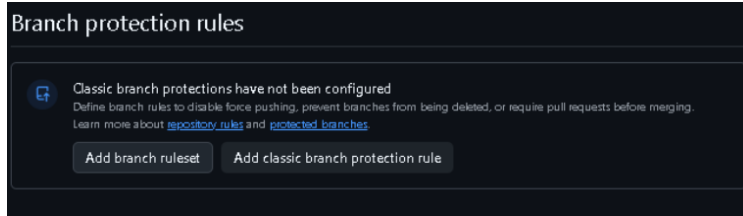
Branches

Branches						New branch
Overview Yours Active Stale All						
Search branches...						
Default						
Branch	Updated	Check status	Behind	Ahead	Pull request	
main	20 hours ago				Default	...
Your branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
feature/springboot	4 hours ago		0	0		...
feature/bd	4 hours ago		0	0		...
feature/README	4 hours ago		0	0		...
develop	4 hours ago		0	0		...
Active branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
feature/springboot	4 hours ago		0	0		...
feature/bd	4 hours ago		0	0		...
feature/README	4 hours ago		0	0		...
develop	4 hours ago		0	0		...

Proteccion de Branches main y develop

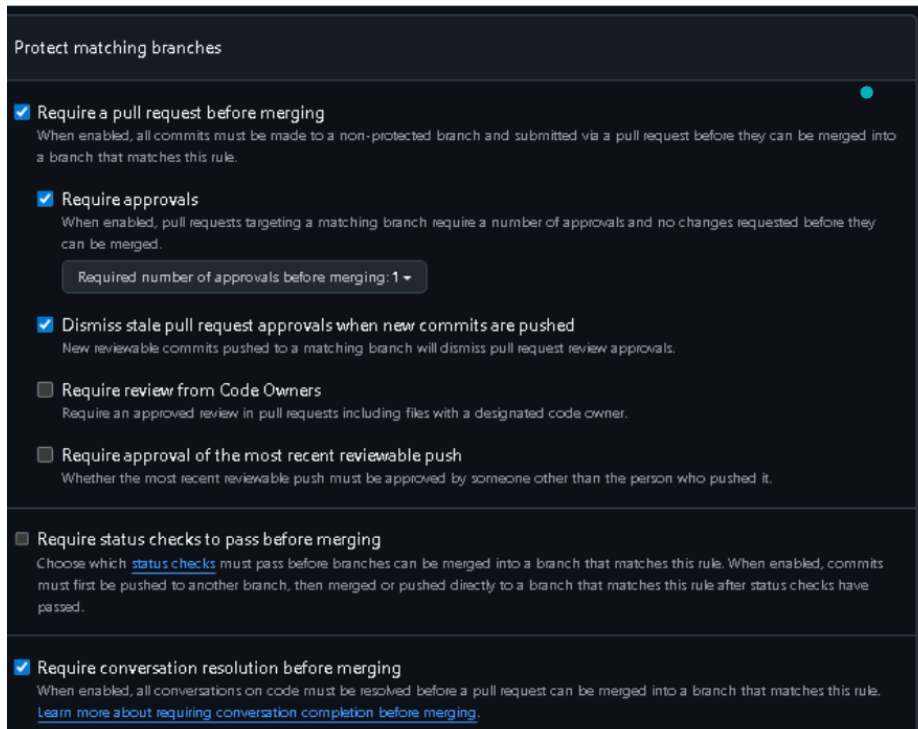
Paso 1:

En configuracion, en branches elegir “add clasisc branch protection rule”

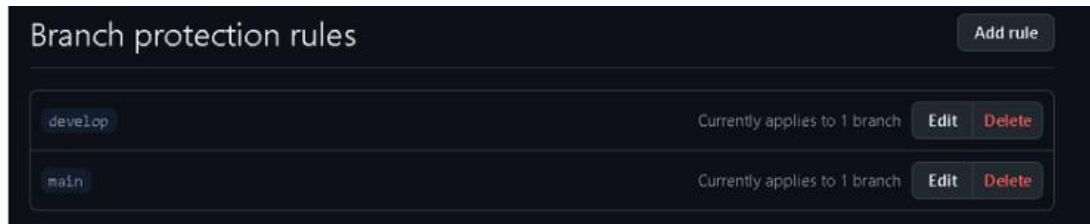


Paso 2:

Se configuran reglas de protección para la rama develop con el fin de garantizar la calidad del código. Estas opciones obligan a que los cambios se realicen mediante pull requests, sean revisados y aprobados antes de integrarse, evitando errores y asegurando un desarrollo más ordenado y colaborativo. Al final seleccionar en “create”



Las ramas se encuentran ahora protegidas



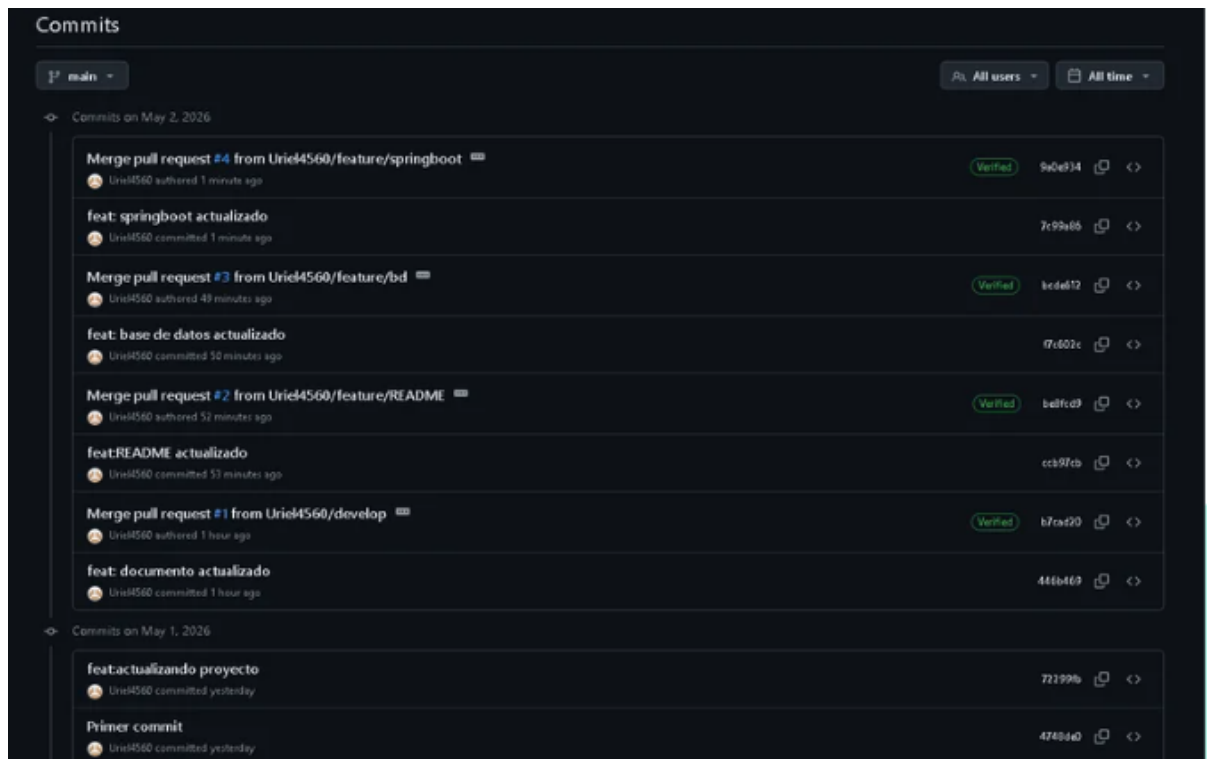
Commits

Un commit es básicamente un registro de cambios guardado en Git.

- ✓ Guardar avances (ej: agregar login, base de datos, etc.)
- ✓ Llevar historial de cambios
- ✓ Poder volver atrás si algo falla
- ✓ Organizar el desarrollo por partes

Existen 2 tipos de commits:

- Commits normales
- Commits de merge (fusión)



- **git log**: Comando que muestra:

- Historial de commits en tu PC
- Autor
- Fecha
- Mensaje del commit
- ID del commit

```

leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (main)
$ git log
commit 72299fb5125e07623d195c9f327fcf7654aab6a2 (HEAD -> main)
Author: Uriel <U23227010@utp.edu.pe>
Date:   Fri May 1 19:43:09 2026 -0500

    feat: actualizando proyecto

commit 4748de09cc5a7e86ecd865f2d3aa4277e94d932f
Author: Uriel <U23227010@utp.edu.pe>
Date:   Fri May 1 13:55:44 2026 -0500

    Primer commit
  
```

- **git log --oneline:** Cada commit en una sola línea muestra:

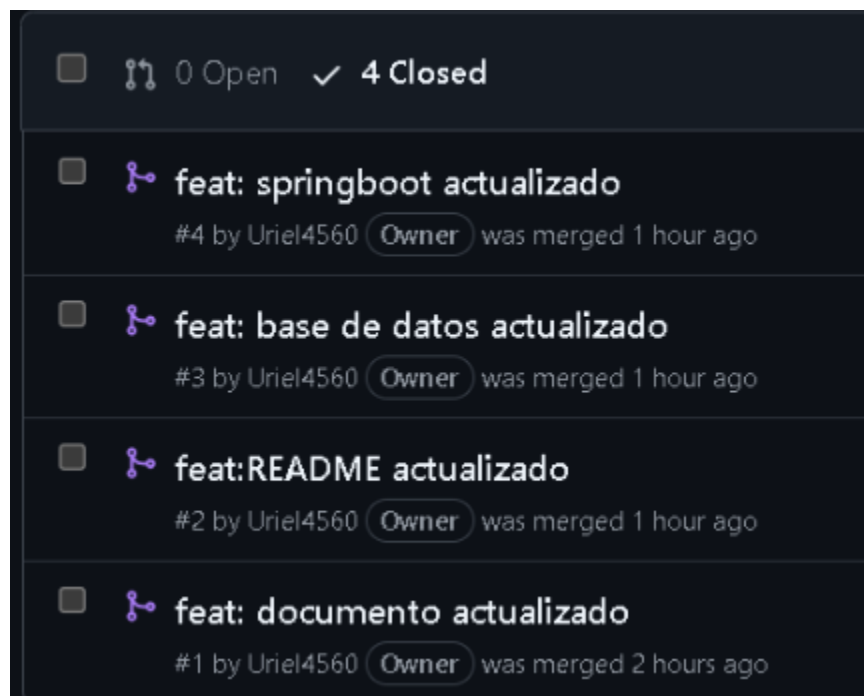
- ID corto del commit
- Mensaje

Pull request

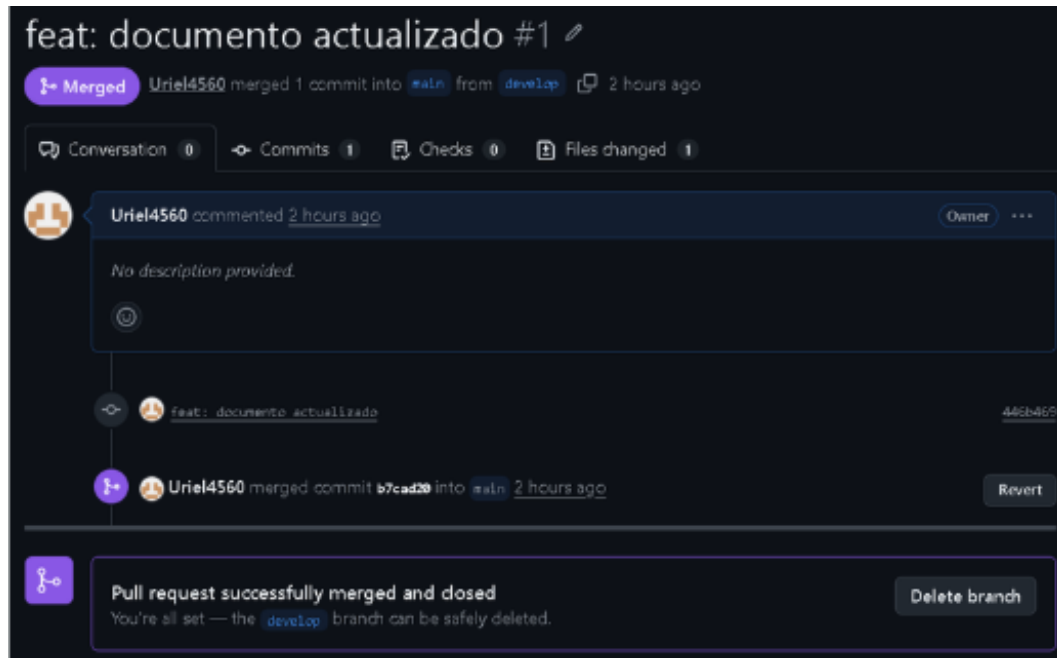
Un Pull Request es una solicitud en GitHub para fusionar cambios de una rama a otra, permitiendo la revisión, discusión y validación del código antes de integrarlo al proyecto.

¿Qué hará en el proyecto?

Permitirá integrar los cambios de cada rama (feature) hacia develop o main de forma controlada.



Ejemplo



Pasos para aplicar merge

1. Ir a la rama destino

- git checkout main

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (main)
$ git checkout main
Already on 'main'
Your branch and 'origin/main' have diverged,
and have 1 and 8 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)
```

- git branch

Muestra las ramas del proyecto y señala en cuál estás trabajando

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (main)
$ git branch
  develop
  feature/README
  feature/bd
  feature/springboot
* main
```

2. Fusionar la rama de trabajo

- `git merge feature/bd`

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow
$ git merge feature/bd
```

3. Subir los cambios a GitHub

- `git push origin main`

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 388 bytes | 388.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Uriel4560/UrbanFlow.git
   9a0e934..c169dd2  main -> main
```

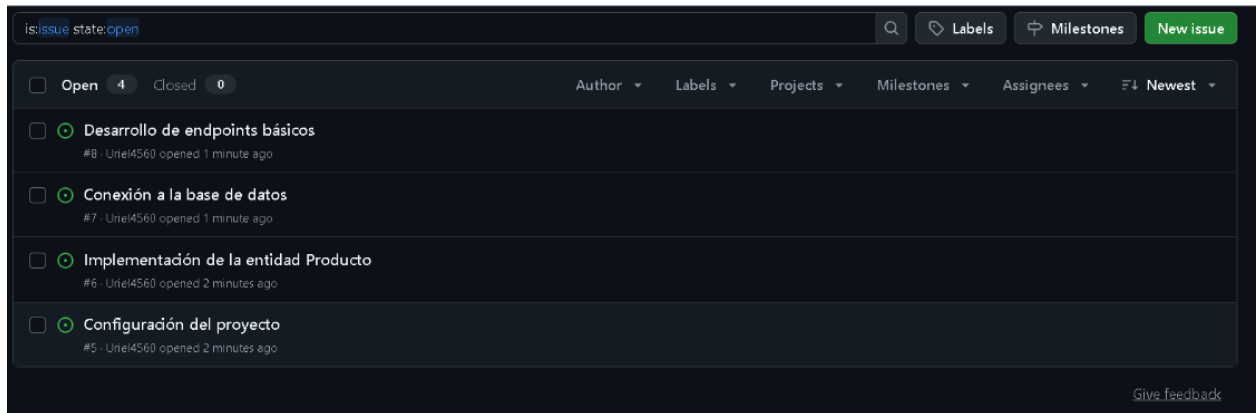
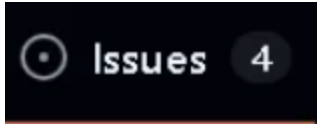
Issues

Los Issues son tareas, problemas o ideas dentro de un proyecto.

¿Para qué sirven?

Se usan para:

- ✓ Reportar errores (bugs)
- ✓ Proponer nuevas funcionalidades
- ✓ Asignar tareas al equipo
- ✓ Llevar seguimiento del trabajo



Configuración de un Issue

- Assignees (Asignados)

Indica quién está encargado del Issue

✓ En este caso elegi: Uriel4560

✓ Esa persona debe resolver la tarea

- Labels (Etiquetas)

Clasifican el tipo de Issue

Ejemplo:

bug → error

feature → nueva funcionalidad

docs → documentación

✓ En este caso: bug (error)

- Projects (Proyecto / Kanban)

Muestra en qué tablero está el Issue

✓ UrbanFlow → tu proyecto

✓ Status: Backlog → está pendiente (aún no se trabaja)

- Milestone

Sirve para agrupar tareas por objetivos o fases

✓ En tu caso: no estoy usando (No milestone)

- Relationships

No se relaciona este Issue con otros

✓ Ej: dependencias o tareas relacionadas

✓ Ahora: vacío

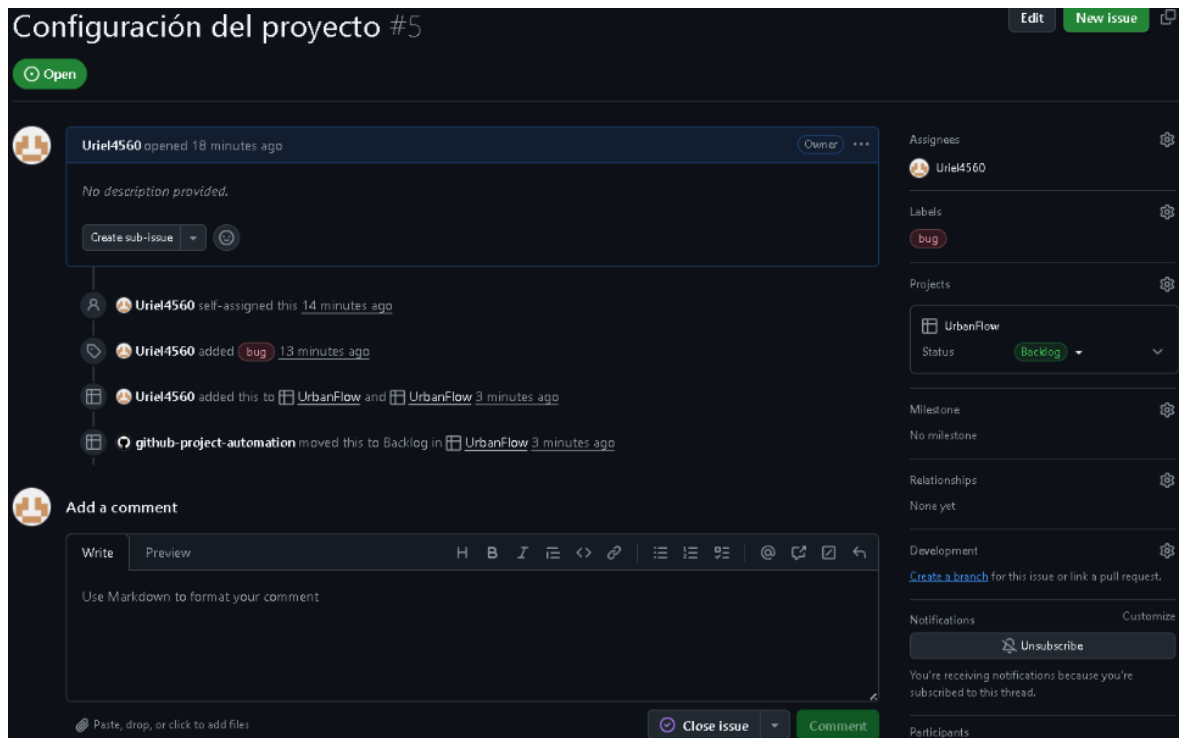
- Development

Permite:

Crear una rama desde el Issue

Vincular un Pull Request

✓ Sirve para conectar código con la tarea



Tablero Kanban

Es una herramienta visual para organizar y controlar tareas de un proyecto.

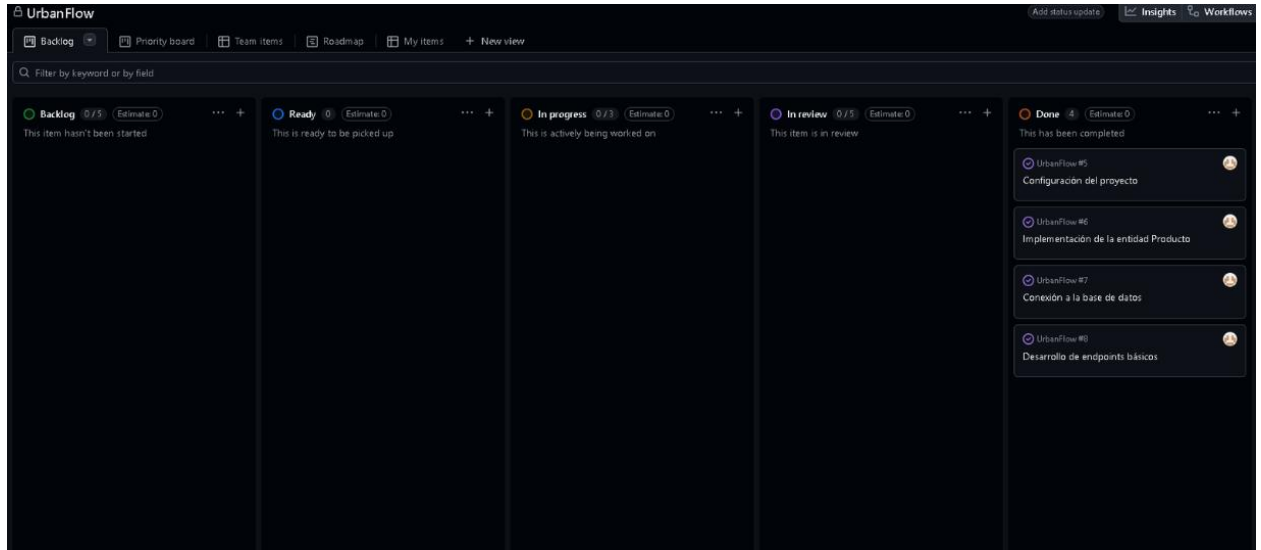
¿Para qué sirve?

- ✓ Organizar el trabajo
- ✓ Ver el progreso del proyecto
- ✓ Mejorar el trabajo en equipo
- ✓ Controlar tareas fácilmente

¿Cómo funciona?

Se divide en columnas, por ejemplo:

- Pendiente (To Do) → tareas por hacer
- En proceso (In Progress) → tareas en desarrollo
- Terminado (Done) → tareas completadas



10. Resultados y Palabras clave

Resultados

El sistema desarrollado cumple con los objetivos planteados, permitiendo gestionar productos, controlar el inventario y registrar pedidos de manera eficiente. Se logró reducir la complejidad de los procesos manuales y mejorar la organización de la información.

Palabras clave

Sistema, Inventario, Pedidos, Spring Boot, MySQL, API REST, Desarrollo, Arquitectura en capas, Ropa.

11. Conclusiones

- El desarrollo del sistema UrbanFlow permitió aplicar herramientas modernas como Spring Boot, MySQL, Git y GitHub para optimizar la gestión de pedidos e inventario en una tienda de ropa, mejorando la organización y el control de la información.
- La implementación de metodologías ágiles como Scrum facilitó el trabajo colaborativo del equipo, permitiendo distribuir tareas, realizar seguimiento del progreso y cumplir los objetivos planteados de manera eficiente.
- El uso de GitHub, ramas, commits, pull requests e issues ayudó a mantener un desarrollo ordenado, seguro y controlado, favoreciendo la integración de cambios y la colaboración entre los integrantes del proyecto.
- El tablero Kanban permitió visualizar el estado de las tareas en cada etapa del proyecto, mejorando la comunicación, la productividad y el seguimiento de las actividades hasta su finalización.